

MA2004B Formula Sheet — Optimizadores

Algoritmo	Update	Param. clave
B. Cíclica	$x_i^{k+1} = x_i^k + \lambda^* e_i$	e_i, ϵ
Powell (conj.)	$x_{k+1} = x_k + \lambda^* d_i; d_{new} = x_n - x_0$	d_i, ϵ
Hooke-Jeeves	$x_{cand} = x^k + \lambda d, d = x^e - x^k$	$\Delta_i, \alpha, \epsilon$
Grad. Desc.	$x_{k+1} = x_k + \lambda_k d_k, d_k = -\nabla f$	λ, ϵ
SGD	$X_{i+1} = X_i - \lambda \nabla f(X)$	λ
Momentum	$V = \beta V - \lambda(1 - \beta) \nabla f; X += V$	λ, β
Nesterov	$\tilde{X} = X + \beta V; V = \beta V - \lambda(1 - \beta) \nabla f(\tilde{X})$	λ, β
AdaGrad	$X_i = X_{i-1} - \frac{\lambda}{\sqrt{r_i}} \nabla f$	λ
RMSProp	$X_i = X_{i-1} - \frac{\lambda}{\sqrt{S_i}} \nabla f$	λ, β
Adam	$X_i = X_{i-1} - \frac{\lambda}{\sqrt{S_i}} \hat{V}_i$	$\lambda, \beta_1, \beta_2$

1. Búsqueda Cíclica

Update:

$$x_i^{k+1} = x_i^k + \lambda^* \cdot e_i$$

Param: e_i =dir. canónicas; λ^* =paso óptimo; ϵ =tol.

Pasos:

1. Punto inicial $x^0 = (x_1^0, \dots, x_n^0)$; dir. $e_1, \dots, e_n$
2. $\forall i$: búsqueda 1D (sección dorada / lineal) sobre e_i que minimiza $f(x^k + \lambda e_i)$
3. Actualizar $x_i^{k+1} = x_i^k + \lambda^* e_i$
4. Verificar parada

1 iter = un barrido por las n variables.

Parada: $\|x^{k+1} - x^k\| < \epsilon$ ó máx iter.

Trigger: búsqueda local sin derivadas, por coordenadas.

2. Powell (Direcciones Conjugadas)

Update:

$$x_{k+1} = x_k + \lambda^* \cdot d_i \\ d_{new} = x_n - x_0$$

Param: $d_1, \dots, d_n$ =direcciones; ϵ =tol.

Pasos:

1. Elegir dir. iniciales $d_1, \dots, d_n$
2. $\forall d_i$: $\min f(x_k + \lambda d_i); x_{k+1} = x_k + \lambda^* d_i$
3. Nueva conjugada $d_{new} = x_n - x_0$; $\min f$ sobre $d_{new} \rightarrow x^*$
4. Reemplazar dir. de mayor disminución por d_{new} ; reiniciar $x_0 = x^*$
5. Verificar parada

1 iter = barrido n dir. + generar conjugada.

Parada: $\|x_{k+1} - x_k\| < \epsilon$ ó máx iter.

Trigger: sin derivadas; direcciones conjugadas.

3. Hooke-Jeeves (Patrón)

Update:

$$d = x^e - x^k, \quad x_{cand} = x^k + \lambda d \\ \Delta_i = \alpha \cdot \Delta_i$$

Param: Δ_i =paso; α =fac. reducción; ϵ =prec.

Pasos:

1. Inicial $x^0, \Delta_i, \alpha, \epsilon$
2. Exploratoria $\forall x_i$: evaluar $f(x_i \pm \Delta_i)$; si mejora $\rightarrow x^e$, si no, mantén x_i
3. Patrón si $f(x^e) < f(x^k)$: $d = x^e - x^k$; $x_{cand} = x^k + \lambda d$; acepta si mejora, si no vuelve a explorar
4. Si no mejora: reducir $\Delta_i = \alpha \Delta_i$

1 iter = exploración + movim. por patrón.

Parada: $\Delta < \epsilon$.

Trigger: local, sin derivadas; explora + patrón.

4. Gradient Descent

Update:

$$d_k = -\nabla f(x_k)$$

$$x_{k+1} = x_k + \lambda_k d_k$$

Param: λ_k =paso (fijo o lin. search); ϵ =tol.

Pasos:

1. Inicial x_0, ϵ, λ_k
2. $d_k = -\nabla f(x_k)$
3. $x_{k+1} = x_k + \lambda_k d_k$
4. (opc.) λ_k por Armijo / Wolfe
5. Verificar parada

1 iter = 1 paso en dirección $-\nabla f$.

Parada: $\|\nabla f(x^*)\| \approx 0$ ($< \epsilon$).

Trigger: 1ª derivada; máx. inclinación.

5. SGD

Update:

$$X_{i+1} = X_i - \lambda \nabla f(X)$$

Param: λ =tasa aprendizaje; $\nabla f(X)$ =grad.

Pasos:

1. Selecciona ejemplo aleatorio / mini-lote
2. Calcula ∇f sólo de ese ejemplo
3. $X_{i+1} = X_i - \lambda \nabla f$ (dir. opuesta)
4. Repetir hasta completar época

$$\text{Mín. cuad.}: f = \frac{1}{2}(\theta_1 + \theta_2 x - y_i)^2;$$

$$\nabla \theta = [(\theta_1 + \theta_2 x - y_i), (\theta_1 + \theta_2 x - y_i)x]^T$$

1 iter = 1 paso con 1 ejemplo/mini-lote.

Trigger: estocástico; evita mín. locales.

6. SGD-Momentum

Update:

$$V = \beta \cdot V - \lambda(1 - \beta) \nabla f(x, y)$$

$$X = X + V$$

Param: λ =tasa; β =coef. momentum.

Pasos:

1. Inicial $X_0, V_0 = 0$
2. $\nabla f(X)$
3. $V = \beta V - \lambda(1 - \beta) \nabla f$
4. $X = X + V$

1 iter = acumula velocidad + paso.

Trigger: inercia; suaviza oscilaciones.

7. Nesterov (NAG)

Update:

$$\tilde{X} = X + \beta \cdot V$$

$$V = \beta \cdot V - \lambda(1 - \beta) \nabla f(\tilde{X})$$

$$X = X + V$$

Param: λ =tasa; β =momentum.

Pasos:

1. Posición anticipada $\tilde{X} = X + \beta V$
2. ∇f evaluado en $\tilde{X}$
3. $V = \beta V - \lambda(1 - \beta) \nabla f(\tilde{X})$
4. $X = X + V$

1 iter = look-ahead + paso.

Trigger: grad. en punto anticipado.

8. AdaGrad

Update:

$$r_i = r_{i-1} + (\nabla f(X))^2$$

$$X_i = X_{i-1} - \frac{\lambda}{\sqrt{r_i}} \cdot \nabla f(X)$$

Param: λ =tasa; r_i =acum. grad².

Pasos:

1. $\nabla f(X)$
2. $r_i = r_{i-1} + (\nabla f)^2$
3. $X_i = X_{i-1} - \frac{\lambda}{\sqrt{r_i}} \nabla f$

1 iter = λ adaptada por parámetro.

Trigger: adaptativo; r crece $\Rightarrow \lambda \downarrow$.

9. RMSProp

Update:

$$S_i = \beta S_{i-1} + (1 - \beta)(\nabla f(X))^2$$

$$X_i = X_{i-1} - \frac{\lambda}{\sqrt{S_i}} \cdot \nabla f(X)$$

Param: λ =tasa; $\beta \in [0.9, 0.99]$.

Pasos:

1. $\nabla f(X)$
2. $S_i = \beta S_{i-1} + (1 - \beta)(\nabla f)^2$ (EMA)
3. $X_i = X_{i-1} - \frac{\lambda}{\sqrt{S_i}} \nabla f$

1 iter = EMA de grad² + paso.

Trigger: adaptativo; EMA evita $\lambda \rightarrow 0$ de AdaGrad.

10. Adam

Update:

$$V_i = \beta_1 V_{i-1} + (1 - \beta_1) \nabla f(X)$$

$$S_i = \beta_2 S_{i-1} + (1 - \beta_2) [\nabla f(X)]^2$$

$$\hat{V}_i = \frac{V_i}{1 - \beta_1^i}, \quad \hat{S}_i = \frac{S_i}{1 - \beta_2^i}$$

$$X_i = X_{i-1} - \frac{\lambda}{\sqrt{\hat{S}_i}} \cdot \hat{V}_i$$

Param: λ ; $\beta_1=0.9$; $\beta_2=0.999$.

Pasos:

1. V_i = EMA grad (Momentum)
2. S_i = EMA grad² (RMSProp)
3. Corrección de sesgo $\hat{V}_i, \hat{S}_i$
4. $X_i = X_{i-1} - \frac{\lambda}{\sqrt{\hat{S}_i}} \hat{V}_i$

1 iter = Momentum + RMSProp + bias-corr.

Trigger: adaptativo + momentum; std deep learning.

EMA (suavizado)

$$S_t = \alpha y_t + (1 - \alpha) S_{t-1}, \quad 0 < \alpha < 1$$

Comparación

Opt	Mom.	Adapt.	Uso
SGD	No	No	Convexos
Momentum	Sí	No	Redes superf.
AdaGrad	No	Sí	Datos dispersos (NLP)
RMSProp	No	Sí	Deep Learning
Adam	Sí	Sí	Aprend. profundo

Adaptativo = λ por parámetro.